

C64 Visual Assembler — User Manual

A visual, block-based 6502 assembler for the Commodore 64. Build programs by dragging and dropping instruction blocks, and see the generated assembly and machine code in real time.

Table of Contents

- 1. [Interface Overview](#)
- 2. [Block Palette](#)
- 3. [Program Area](#)
- 4. [ASM View](#)
- 5. [Settings & Toolbar](#)
- 6. [Addressing Modes](#)
- 7. [Standard 6502 Instructions](#)
- 8. [Macro Blocks — Reference](#)
 - [LABEL](#)
 - [COMMENT](#)
 - [BYTE](#)
 - [WORD](#)
 - [FILL](#)
 - [ALIGN](#)
 - [TEXT](#)
 - [STRING](#)
 - [DATA](#)
 - [RAWBYTES](#)
 - [RAWTEXT](#)
 - [INCBIN](#)
 - [SID](#)
 - [INCLUDE](#)
 - [TABLE](#)
 - [LOOP / NEXT](#)
 - [PUSH / PULL](#)
 - [MACRO / ENDM / INVOKE](#)
 - [DEFINE / IF / ELSE / ENDIF](#)
- 9. [Knowledge Base Links](#)

1. Interface Overview

The app is split into three main panels:

Panel	Description
Left — Palette	All available instruction and macro blocks. Search or browse by category.
Center — Program	Your program. Drag blocks here, reorder them, edit operands.

Panel	Description
Right — Output	Live ASM view and/or memory monitor output.

2. Block Palette

The palette on the left lists all available blocks grouped by category:

- **Data movement** — LDA, LDX, STA, STX, ...
- **Arithmetic** — ADC, SBC, INC, DEC, CMP, ...
- **Logic** — AND, ORA, EOR, BIT
- **Jumps & Branches** — JMP, JSR, RTS, BNE, BEQ, ...
- **Register operations** — TAX, TAY, INX, DEX, ...
- **Shift & Rotate** — ASL, LSR, ROL, ROR
- **Stack** — PHA, PHP, PLA, PLP
- **System** — CLC, SEC, NOP, BRK, ...
- **Illegal instructions** — LAX, SAX, DCP, ...
- **Structure** — LABEL, COMMENT
- **Macros** — LOOP, NEXT, PUSH, PULL, TEXT, BYTE, WORD, FILL, ALIGN, STRING, DATA, RAWBYTES, RAWTEXT, INCBIN, SID, INCLUDE, TABLE, MACRO, ENDM, INVOKE, IF, ELSE, ENDIF

Use the **search box** at the top of the palette to filter by name. Click the **Add selected block** button or drag a block into the program area.

3. Program Area

- **Drag & drop** blocks from the palette, or **reorder** existing blocks by dragging their handle (≡).
- Each block shows its **mnemonic**, **operand field**, and **addressing mode selector** (where applicable).
- Click the ▶ / ▼ toggle to collapse or expand a block.
- Use the × (**delete**) button on a block to remove it.
- **Collapse All** button folds all blocks at once.

Operand input

- For branch/jump instructions (**BNE**, **JMP**, **JSR**, etc.) a **label picker** dropdown appears — click a defined label to insert it.
- Number format follows the **HEX / DEC** toggle in the toolbar (see section 5).

4. ASM View

The right panel shows the generated output in real time.

Output modes

Mode	Description
ASM	6502 assembly source with addresses and labels

Mode	Description
Monitor	Hex / byte dump (C64 monitor style)
Both	ASM on top, monitor below

HEX / DEC toggle (ASM view)

A small HEX / DEC toggle next to the *Macro source* checkbox switches all address and value literals in the ASM output between hexadecimal and decimal.

Macro source

When enabled, macro definition blocks (MACRO...ENDM) show their source code inline in the ASM view.

Clicking an ASM line

Click any line in the ASM view to **highlight the corresponding block** in the program area.

Origin

Set the load address of your program in the **Origin** field (default `$0801`). The address updates all label addresses and the monitor output.

5. Settings & Toolbar

Control	Description
Number base (HEX / DEC)	Sets the display/input format for operands throughout the UI
Language	Switch between English and Hungarian
Theme	Light / dark mode
BASIC SYS stub	Prepends a BASIC line that calls SYS to your program's origin
Sample	Load a built-in example program
Zoom in / out	Scale the block UI (affects all block elements)
Save Project	Save the current program as a <code>.json</code> project file
Load Project	Load a previously saved project
Save PRG	Export the compiled binary as a <code>.prg</code> file
Run in VICE	Compile and launch directly in the VICE emulator
Clear program	Remove all blocks from the program area
Collapse All	Collapse all blocks
About	Version info
What's New	Changelog

Control	Description
Knowledge Base	Reference links (6502 opcodes, C64 KERNAL, memory map, colors)
Check for Update	Open the itch.io page to check for a newer release

6. Addressing Modes

Each 6502 instruction supports one or more addressing modes. The mode selector appears on each block.

Mode	Label	Example	Description
implied	Implied	NOP	No operand; the instruction is self-contained
immediate	Immediate	LDA #\$FF	Inline constant; the assembler adds # automatically
zeroPage	Zero page	LDA \$10	Single byte address in page zero (0–255)
absolute	Absolute	LDA \$0400	Full 16-bit memory address
relative	Relative/Label	BNE loop	For branch instructions; enter a label name or target address
absoluteX	Absolute,X	LDA \$0400,X	16-bit address + X register offset
absoluteY	Absolute,Y	LDA \$0400,Y	16-bit address + Y register offset
indirectX	Indirect,X	LDA (\$FB,X)	Zero page indexed indirect
indirectY	Indirect,Y	LDA (\$FB),Y	Zero page indirect indexed
indirect	Indirect	JMP (\$0100)	Indirect; only usable with JMP
zeroPageY	Zero page,Y	LDX \$FB,Y	Zero page address + Y register offset

7. Standard 6502 Instructions

Data Movement

Mnemonic	Description	Modes
LDA	Load Accumulator	immediate, zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
LDX	Load X register	immediate, zeroPage, zeroPageY, absolute, absoluteY
LDY	Load Y register	immediate, zeroPage, absolute, absoluteX

Mnemonic	Description	Modes
STA	Store Accumulator	zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
STX	Store X register	zeroPage, zeroPageY, absolute
STY	Store Y register	zeroPage, absolute

Arithmetic

Mnemonic	Description	Notes
ADC	Add with carry	Set carry with SEC before use in most cases
SBC	Subtract with carry	Set carry with SEC before subtraction
INC	Increment memory	—
DEC	Decrement memory	—
CMP	Compare with A	Sets flags; does not modify A
CPX	Compare with X	—
CPY	Compare with Y	—

Logic

Mnemonic	Description
AND	Logical AND with Accumulator
ORA	Logical OR with Accumulator
EOR	Exclusive OR with Accumulator
BIT	Test bits in memory against A (sets N, V, Z flags)

Jumps & Branches

Mnemonic	Description
JMP	Unconditional jump (absolute or indirect)
JSR	Jump to subroutine (saves return address on stack)
RTS	Return from subroutine
RTI	Return from interrupt
BNE	Branch if Not Equal (Z=0)
BEQ	Branch if Equal (Z=1)
BCC	Branch if Carry Clear (C=0)

Mnemonic	Description
BCS	Branch if Carry Set (C=1)
BMI	Branch if Minus (N=1)
BPL	Branch if Plus (N=0)
BVC	Branch if Overflow Clear (V=0)
BVS	Branch if Overflow Set (V=1)

Register Operations

Mnemonic	Description
TAX	Transfer A → X
TAY	Transfer A → Y
TXA	Transfer X → A
TYA	Transfer Y → A
TSX	Transfer Stack pointer → X
TXS	Transfer X → Stack pointer
INX	Increment X
DEX	Decrement X
INY	Increment Y
DEY	Decrement Y

Shift & Rotate

Mnemonic	Description
ASL	Arithmetic Shift Left
LSR	Logical Shift Right
ROL	Rotate Left through Carry
ROR	Rotate Right through Carry

Stack

Mnemonic	Description
PHA	Push Accumulator onto stack
PHP	Push Processor status onto stack
PLA	Pull Accumulator from stack

Mnemonic	Description
PLP	Pull Processor status from stack

System / Flags

Mnemonic	Description
CLC	Clear Carry flag
CLD	Clear Decimal mode
CLI	Clear Interrupt disable
CLV	Clear Overflow flag
SEC	Set Carry flag
SED	Set Decimal mode
SEI	Set Interrupt disable
NOP	No operation
BRK	Force break / software interrupt

Illegal / Undocumented Instructions

These are supported for advanced use. Use with care — behavior may differ between chips.

LAX, SAX, DCP, ISC, SLO, RLA, SRE, RRA, ANC, ALR, ARR, AXS

8. Macro Blocks — Reference

Macro blocks generate multiple instructions or data directives automatically. They are found in the **Macros** category of the palette.

LABEL

Creates a named label that can be used as a jump target.

Field	Description
Label name	Identifier used in JMP , JSR , BNE , etc.

Generated ASM:

```
loop:  ; $0820
```

The address is shown as a comment. Labels have **0 byte size**.

COMMENT

Inserts a comment line in the ASM output. No bytes generated.

Generated ASM:

```
; Your comment text here
```

BYTE

Inserts an arbitrary sequence of raw bytes.

Field	Description
Operand	Comma-separated byte values (e.g. <code>\$01</code> , <code>\$02</code> , <code>\$FF</code> or <code>1</code> , <code>2</code> , <code>255</code>)

Generated ASM:

```
.byte $01, $02, $FF
```

Size: Number of bytes in the list.

WORD

Inserts 16-bit values stored as little-endian LO/Hi byte pairs.

Field	Description
Operand	Comma-separated 16-bit values (e.g. <code>\$0400</code> , <code>\$C000</code>)

Generated ASM:

```
.word $0400, $C000
```

Size: 2 bytes per word.

FILL

Generates a repeated sequence of the same byte value.

Field	Description
Operand	<code>count,value</code> — e.g. <code>256,0</code> fills 256 bytes with zero

Generated ASM:

```
.fill 256, $00
```

Size: The count value in bytes.

ALIGN

Advances the program counter to the next boundary of the given size. Inserts padding bytes as needed.

Field	Description
Boundary	Alignment value — e.g. 64 (sprite boundary), 256 (page), \$2000 (bitmap)

Generated ASM:

```
; ALIGN 64 → $0840 (12 bytes padding)
```

Size: Dynamic — depends on the current program counter position.

Tip: Use `ALIGN 64` before sprite data, `ALIGN 256` to ensure page-aligned tables.

TEXT

Writes a text string to the C64 screen RAM at a given row/column position using the KERNAL CHROUT routine.

Field	Description
Text	The string to display
X	Column (0–39)
Y	Row (0–24)

Generated ASM:

```
LDA #$48      ; 'H'
JSR $FFD2
LDA #$45      ; 'E'
JSR $FFD2
...
```

Characters are encoded as PETSCII. **Size:** `text.length × 5` bytes (LDA + JSR per character).

STRING

Places a PETSCII-encoded string as raw bytes at a given memory address (no runtime code, deferred data section).

Field	Description
Text	The string to place
Address	Target memory address (e.g. <code>\$C000</code>)

Generated ASM:

```
.byte $48, $45, $4C, $4C, $4F ; "HELLO"
```

Placed at the specified address. **Size:** `text.length` × 5 bytes (counted at destination).

DATA

Writes raw bytes to a given memory address using LDA/STA instructions at runtime.

Field	Description
Bytes	Comma-separated byte values
Address	Target memory address

Generated ASM:

```
LDA #$01
STA $C000
LDA #$02
STA $C001
...
```

Size: `byte_count` × 5 bytes (one LDA + one STA per byte).

RAWBYTES

Places raw bytes at a given memory address — no runtime code is generated. The bytes appear in the deferred data section of the output.

Field	Description
Bytes	Comma-separated byte values
Address	Target memory address (e.g. <code>\$C000</code>)

Size in code: 0 bytes. The data is placed at the given address in the output.

Use this instead of DATA when you don't need runtime initialization code and just want bytes at an address.

RAWTEXT

Like RAWBYTES but accepts a text string that is encoded as PETSCII bytes.

Field	Description
Text	String to encode
Address	Target memory address

Size in code: 0 bytes.

INCBIN

Includes an external binary file and places its content at a given memory address.

Field	Description
File	Browse to select a <code>.bin</code> , <code>.prg</code> , <code>.sid</code> , or <code>.raw</code> file
Address	Target load address (e.g. <code>\$C000</code>)

Generated ASM comment:

```
; INCBIN "music.bin" @ $C000 (2048 bytes)
.byte $01, $02, ...
```

Size in code: 0 bytes (deferred data section). The binary is embedded at the given address.

SID

Loads a SID music file and places its data at the address specified in the SID header. Header metadata (Load/Init/Play addresses, title, author) is extracted automatically.

Field	Description
File	Browse to select a <code>.sid</code> file

The block displays:

- **Title / Author** from the SID header
- **Load address** — where the data is placed in memory
- **Init address** — call this with JSR to initialize the music

- **Play address** — call this with JSR on every frame (e.g. in an IRQ handler)

Generated ASM comment:

```
; SID "Commando.sid" @ $1000  Init:$1000  Play:$1003  (8192 bytes)
```

Size in code: 0 bytes. The SID data is placed at the load address.

Typical usage: JSR to Init once, then call Play periodically from an IRQ.

INCLUDE

Includes another Visual Assembler project file and expands its blocks inline at this position. The included blocks are read-only.

Field	Description
File	Browse to select a <code>.json</code> Visual Assembler project

Generated ASM:

```
; === INCLUDE "library.json" - 12 block(s) ===  
... (expanded blocks follow)
```

Tip: Use INCLUDE to build reusable subroutine libraries that you can share across projects.

TABLE

Defines a lookup table at a fixed memory address with a named label.

Field	Description
Name	Label identifier for the table (e.g. <code>color_table</code>)
Address	Fixed address where the table starts (e.g. <code>\$C000</code>)

Generated ASM:

```
color_table:
```

The program counter jumps to the specified address for subsequent blocks. Place BYTE/WORD/FILL blocks after TABLE to fill the table content.

Size: 0 bytes.

LOOP / NEXT

A counted loop pair using a register as the counter.

LOOP

Field	Description
Register	X or Y — the counter register
Count	Loop iteration count (hex or decimal, e.g. 0A = 10)
Label	Auto-generated loop label (e.g. loop0)

Generated ASM:

```
LDX #$0A
loop0:
```

Size: 2 bytes (LD_ opcode + immediate operand).

NEXT

Field	Description
Register	Automatically matched to the LOOP register
Label	Automatically linked to the LOOP label

Generated ASM:

```
DEX
BNE loop0
```

Size: 3 bytes (DEX + BNE + branch offset).

Example — clear 10 screen cells:

```
LDX #$0A
loop0:
LDA #$20      ; space character
STA $0400,X
DEX
BNE loop0
```

PUSH / PULL

Save and restore registers using the stack.

PUSH

Pushes one or more registers onto the stack. The order is always $A \rightarrow X \rightarrow Y$ (innermost first).

Field	Description
Registers	Any combination: <code>A</code> , <code>X</code> , <code>Y</code> , <code>AX</code> , <code>AY</code> , <code>XY</code> , <code>AXY</code>

Generated ASM (example: `AX`):

```
PHA
TXA
PHA
```

Size: 1 byte for A (`PHA`), 2 bytes for X or Y (transfer + push).

PULL

Restores registers from the stack in reverse order ($Y \rightarrow X \rightarrow A$).

Field	Description
Registers	Same as PUSH — must match the corresponding PUSH block

Generated ASM (example: `AX`):

```
PLA
TAX
PLA
```

Important: Always pair PUSH and PULL with the same register set, and in reverse order. `PUSH AX` must be matched by `PULL AX` (which internally does Y-first, then X, then A).

MACRO / ENDM / INVOKE

Define and reuse your own named code blocks.

MACRO (definition start)

Field	Description
Name	Identifier for the macro (e.g. <code>clear_screen</code>)

Marks the beginning of a macro definition. All blocks between MACRO and ENDM become the macro body. The definition does **not** generate code where it appears.

Generated ASM:

```
; .MACRO clear_screen
    ... (body blocks)
; .ENDM
```

ENDM (definition end)

Closes the current macro definition. No fields.

INVOKE

Inserts the contents of a named macro at this position.

Field	Description
Macro name	Select from the dropdown of defined macros

Generated ASM:

```
; >>> Invoke: clear_screen
    ... (expanded macro body)
```

The macro body is expanded inline — all addresses and labels are resolved in context.

Tip: Define macros at the top (or bottom) of your program, then INVOKE them wherever needed. Macros can be invoked multiple times.

DEFINE / IF / ELSE / ENDIF

Conditional assembly blocks. Add a **DEFINE** block to activate a named symbol — any **IF** block whose condition matches a **DEFINE** present in the program is assembled; its **ELSE** branch (if any) is skipped, and vice versa.

DEFINE

Field	Description
Symbol	One or more comma-separated identifiers to activate (e.g. DEBUG or DEBUG, PAL)

Generated ASM:

```
; .DEFINE DEBUG
; .DEFINE DEBUG, PAL
```

A single **DEFINE** block can activate multiple symbols at once using a comma-separated list. Place **DEFINE** blocks anywhere in the program (typically at the top). Removing the block deactivates all its symbols instantly.

IF

Field	Description
Condition	Identifier to test (must match a DEFINE symbol to be active)

Generated ASM:

```
; .IF DEBUG
```

Blocks between **IF** and **ENDIF** (or **ELSE**) are included or skipped based on whether the condition symbol has a matching **DEFINE** block in the program. Skipped blocks appear as `; [IF skipped] ...` comments and generate **zero bytes**.

ELSE

No fields. Marks the alternative branch — assembled when the **IF** condition is *not* active.

Generated ASM:

```
; .ELSE
```

ENDIF

No fields. Closes the conditional block.

Generated ASM:

```
; .ENDIF
```

Size: 0 bytes for all four blocks. Only the content *between* them counts.

Example — debug border flash, release build skips it:

```
; .DEFINE DEBUG

; .IF DEBUG
    LDA #$02      ; red border
    STA $D020
; .ELSE
    LDA #$00      ; black (release)
```



```
    STA $D020
; .ENDIF
```

Example — multiple symbols in one DEFINE block:

```
; .DEFINE DEBUG, PAL

; .IF PAL
    LDA #$xx      ; PAL timing constant
; .ELSE
    LDA #$xx      ; NTSC timing constant
; .ENDIF
```

Nested **IF** blocks are supported. The innermost condition is evaluated independently; an outer skipped block causes all inner blocks to be skipped regardless of their condition.

9. Knowledge Base Links

Quick reference links available in the app under **Knowledge Base**:

Resource	URL
6502 Opcodes Reference	http://www.6502.org/tutorials/6502opcodes.html
C64 KERNAL Functions	https://sta.c64.org/cbm64krnfunc.html
C64 Memory Map	https://sta.c64.org/cbm64mem.html
C64 Color Codes	https://sta.c64.org/cbm64col.html